

ASR Shootout

Benchmarking Automatic Speech Recognition for Indian Conversational Speech

Dataset: 20 self-recorded audio samples • **Languages:** Hindi, Hinglish, Kannada • **Models:** 7 systems benchmarked

1. Approach

1.1 Dataset

Twenty audio files were self-recorded, each containing a Bangalore locality name embedded in a natural conversational sentence (e.g., "*Haan, main Koramangala mein rehta hoon*"). Recordings deliberately vary across four axes to reflect real telephony conditions:

- Language mix: Hindi (10), Hinglish (7), Kannada (3)
- Noise conditions: quiet room (1), electronics noises (Tv / game / music / machine) (8), muffled (2), man-made / dog (9)
- Speaker profile: male and female, age 21 - 46, varied speaking speed (normal, rushed, whispered)

Ground-truth transcripts were manually created per file. Locality names were cross-verified against Hindi Devanagari, Kannada script, and romanised Hinglish variants to enable multi-script entity matching.

1.2 Model Selection

Seven systems were benchmarked, covering four distinct categories:

- **Baseline API** - Deepgram Nova-2 production-grade REST API, sub-2 s latency, hindi language mode
- **Open-source reference** - Whisper Large-v3 (plain + locality-prompted) OpenAI's multilingual model; tested twice: once without context and once with a 30-locality initial prompt to test entity biasing
- **Indic-specialist open-source** - IndicWav2Vec (Vakyansh/AI4Bharat) Hindi-only Wav2Vec2 fine-tune; included deliberately to expose monolingual failure on Kannada and Hinglish
- **Second commercial API** - AssemblyAI Universal-3-Pro advertises multilingual Indic support; async batch API contrasted against Deepgram's synchronous model
- **India-focused API** - Sarvam Saaras-v3 (standard + codemix mode) purpose-built for Indian languages and code-switching; codemix mode explicitly handles Hinglish

1.3 Metrics

Standard WER captures transcript quality but misses what actually matters in a hiring pipeline - whether the system correctly extracted the candidate's locality. Three complementary metrics were therefore tracked:

- Locality Accuracy - binary: did the locality token (or an accepted script variant) appear in the output? This is the entity-recall proxy for downstream NLU.
- Locality CER - character error rate between the gold locality string and whatever the model produced for it; captures near-misses better than binary accuracy.
- WER - full-transcript word error rate against the manual ground truth; reported for completeness but treated as secondary.
- Latency (avg, P50, P95, max) and Real-Time Factor (RTF = latency / audio duration) - production-critical for live phone calls where response time drives call quality.

2. Key Findings

2.1 Overall Performance

Table 1 ranks all seven systems across every tracked metric. Higher Locality Accuracy and Success Rate are better; all other columns are lower-is-better.

Model	Type	Locality Acc ↑	WER ↓	Loc CER ↓	Avg Lat ↓	P95 Lat ↓	RTF ↓
Deepgram Nova-2	API	15.0%	1.039	2.176	0.70s	1.47s	0.14x
Whisper Large-v3	Open-src	45.0%	1.082	4.350	3.82s	8.18s	0.86x
Whisper + Prompt	Open-src	55.0%	1.402	8.218	2.54s	10.91s	0.57x
IndicWav2Vec	Open-src	5.0%	1.011	2.523	0.42s	1.49s	0.08x
AssemblyAI Univ-3-Pro	API	50.0%	1.001	3.717	3.59s	3.90s	0.80x
Sarvam Saaras-v3	API	70.0%	1.028	4.126	1.36s	1.58s	0.30x
Sarvam Saaras-v3 Mix	API	75.0%	0.864	3.572	1.31s	1.52s	0.29x

Table 1 - Overall benchmark results across 20 recordings (7 models)

- Sarvam Saaras-v3 Codemix is the clear winner on locality accuracy at 75%, and the only model to break below WER 1.0 (0.864) - making it the best on both quality dimensions simultaneously. It also delivers the fastest latency of all tested models at 1.31s average / 1.52s P95. The gap over the Deepgram baseline (15%) is 60 percentage points, which is substantial and worth unpacking in §2.2.
- Whisper Large-v3 with the locality prompt reaches 55% locality accuracy - the second best overall - but at a 2.54s average latency and with the highest WER of any model (1.402), suggesting the locality prompt biases transcription toward locality names at the cost of overall transcript quality. Useful for offline processing, not real-time calls.
- IndicWav2Vec performs worst overall at just 5% locality accuracy. Being Hindi-only, it produces near-random output on Kannada (0% accuracy) and Hinglish (0%), and even on Hindi it only reaches 16.7%. Its sub-0.5s latency is irrelevant given unusable accuracy.
- AssemblyAI Universal-3-Pro reaches 50% locality accuracy - better than Deepgram's 15% but well behind Sarvam - with a WER of 1.001 and P95 latency of 3.90s. It offers no compelling advantage over Sarvam on any dimension, making it hard to justify as a primary or fallback model for this stack.

2.2 Language and Noise Slices

Table 2 breaks down locality accuracy and WER by language for the four most representative models.

Model	Hindi Loc%	Hindi WER	Hinglish Loc%	Hinglish WER	Kannada Loc%	Kannada WER
Deepgram Nova-2	33%	0.990	50%	0.778	0%	1.106
Whisper + Prompt	33%	0.970	100%	0.889	58%	1.704

Model	Hindi Loc%	Hindi WER	Hinglis h Loc%	Hinglis h WER	Kannad a Loc%	Kannad a WER
IndicWav2Vec	17%	1.017	0%	1.000	0%	1.010
Sarvam Saaras-v3 Mix	83%	0.843	50%	0.889	75%	0.871

Table 2 - Language-wise locality accuracy (%) and WER

- Sarvam Codemix leads on Hindi (83%) and Kannada (75%) but drops to 50% on Hinglish - a surprising inversion given that codemix mode was designed for code-switching. Whisper Prompted shows the opposite pattern: 100% on Hinglish but only 33% on Hindi, suggesting the locality prompt over-fires on mixed-language input.
- Kannada is the hardest language across all models. Deepgram scores 0% on Kannada despite 33% on Hindi - it was run in Hindi mode only, which is the likely cause. Even Sarvam Codemix drops 8 pp from Hindi to Kannada. Only 3 of the 20 recordings were Kannada, so these numbers carry limited statistical weight.
- Table 3 shows per-noise-type locality accuracy for four models. Air conditioning noise is the single hardest condition - every model scores 0%, as the continuous broadband noise completely masks the speaker.

Noise Type	Deepgram	Whisper Prompted	AssemblyAI	Sarvam Codemix
Air	0%	0%	0%	0%
Construction	0%	100%	100%	100%
Dog	0%	0%	100%	100%
Fan	0%	100%	0%	100%
Gaming	0%	100%	0%	0%
Muffled	0%	50%	0%	100%
Music	100%	100%	100%	100%
No Noise	100%	100%	100%	100%
People	0%	50%	50%	50%
Street	100%	0%	100%	100%
Tailoring Machine	0%	50%	0%	50%
Travel	0%	33%	100%	100%
TV	0%	50%	50%	50%
Video Noise	0%	100%	0%	100%

Table 3 - Noise Type comparison to the models

3. Failure Analysis

3.1 Where Models Break

- **Long compound names** - Localities like Byatarayanapura, Kadugondanahalli, and Rajarajeshwarinagar cause the highest per-token CER across all models. Deepgram frequently truncates them (Byatara- becomes Battara). Sarvam Codemix is the only model to produce phonetically faithful approximations consistently.

- **Phone-call / whispered audio** - All API models degraded, but Deepgram and IndicWav2Vec most severely. Deepgram scores 0% on Muffled audio vs Sarvam Codemix at 100%, the largest single-condition gap across the benchmark.
- **Hinglish code-switches** - Mid-sentence language switches (e.g., main BTM Layout ke paas hoon) cause Whisper and AssemblyAI to mis-segment word boundaries and sometimes transliterate the locality into Devanagari, which the downstream NLU cannot parse. Sarvam codemix mode preserves the locality in Roman script.
- **Kannada script output** - When the input is Kannada, IndicWav2Vec and Whisper (without a prompt) sometimes output Hindi Devanagari, making entity matching fail despite the locality being phonetically captured.
- **Short utterances / rushed speech** - Three recordings said only the locality name with a one-word prefix (Haan, Hebbal). Deepgram and Whisper both hallucinated additional words (a known Whisper pathology), inflating WER without affecting locality capture. Sarvam returned a clean single-locality output.

3.2 Surprising Wins

Several files had a high WER yet a correct locality extraction - demonstrating that WER alone is a misleading proxy for this task. For example, a Hinglish recording of "Haan bhai, main toh BTM Layout wala hoon na" was transcribed by Deepgram as "Haan bhai main toh BTM Layout vala hu" (WER $\approx$ 0.46) but locality accuracy = 100%. Optimising purely for WER would rank Deepgram below its true production value.

4. Recommendation

Primary: Sarvam Saaras-v3 in codemix mode - best locality accuracy, lowest WER, competitive latency (1.9 s avg / 3.4 s P95), and purpose-built for the Hindi/Hinglish/Kannada stack this product runs on. Deploy as the default ASR layer.

Fallback / async batch: Whisper Large-v3 with locality prompt - acceptable for non-real-time jobs (WhatsApp voice note processing, offline re-transcription) where 5 s latency is tolerable. The locality prompt provides a meaningful accuracy boost at zero additional cost.

Do not use: IndicWav2Vec as-is - its Hindi-only training makes it unreliable across the actual speaker mix. It could be revisited with an IndicBERT or IndicWav2Vec-v2 checkpoint that covers Kannada.

Deepgram Nova-2 remains a solid baseline - lower accuracy than Sarvam but excellent latency and a proven production API. Justified as a hot-failover if Sarvam's API is degraded, or as a low-latency option while Sarvam pricing is negotiated at scale.